# 1. Overview and Goals

The goal of this assignment was to create a collection of Java image effects in the form of array traversal, subclass creation, and testing. The images were represented as 2-D arrays of pixels which took integers. Each pixel had red, green, and blue components that went from 0 to 255 in value.

Basically what I did was I completed the rest of the required effects that existed in transformations.java. The Invert effect was already completed and given to me and it was part of the starter code which I did not touch or manipulate in any way during this entire process.

My goals in this project was to have correct, functioning code that had good clarity and would give me the right effects when I passed a test image. Basically, each class does one transformation through the apply method and instead of just changing the color values directly, I used the getRed, getGreen, getBlue and makePixel methods.

# 2. Solution Design

Most of the effects utilized nested for loops to visit every pixel once. I traversed through the width and the height: the outer loop tracks the column (x) and the inner loop tracks the row (y), so in order to access each pixel, I had to call pixels[y][x]. So for the color channels, when I looked at the “OnlyColor” effects, I had to preserve one component and then set the other two to zero: this was done using makePixel. The “NoColor” effects, I had to set that color to zero and keep the other two the same.

For BlackAndWhite, I computed the integer average of the red, green, and blue and then I assigned that to each parameter in makePixel. The result is actually gray, but for my purpose, Black + White = Gray, and when I tested it, it did indeed show up as gray.

For Threshold, each color is evaluated separately. A channel becomes 255 when the original value was greater than the selected threshold, and becomes 0 if it is less than the threshold. The output is limited to eight possible RGB colors, and any value exactly equal to the threshold also becomes 0.

Vertical Reflect swaps the pixels across the left and the right halves of each row, while HorizontalReflect swaps pixels across the top and bottom halves of each column. The loop condition was width/2 and height/2 because it would swap each value and make its way to the center;

(Ex: 1 2 3 4 5 6 7 -> 1 and 7 swap, 2 and 6 swap, and 3 and 5 swap. So for an array of length n, it makes n/2 swaps. )

Growth created an output with twice the height and width. Each output location reads from pixels[y/2][x/2] and that's only one pixel. If we are trying to put one pixel onto 4 pixels, we need to find the other pixels surrounding it, and for that I had to to put the pixel onto pixels[y*2][x*2], pixels[y*2][x*2+1], pixels[y*2+1][x*2], and pixels[y*2+1][x*2+1].

## 3. Discussion and Evaluation

My completed program includes all twelve of the required effects as well as the example Invert effect. The program works with standard rectangular images. Therefore, you do not need a square image or one with even dimensions.

Most of the effects transform the current pixel grid. Grow and Shrink must make new grids, as their results are different sizes. I assume the image is not blank and that each row contains the same number of pixels. I do not check to ensure the grid is valid so my program may crash on uneven images.

Odd widths and heights are not a problem with the color effects, reflections, Threshold, or Grow. Shrink was the one which was affected by the odd condition: if there is an extra row or column that cannot make up a group of four pixels, that side is cropped off the image that is generated. If your image was less than 2-by-2

pixels you may end up with an output that has 0 rows or columns. The thing I learned that was most interesting was that big visual effects can be made with tiny rules. Dropping one number from each pixel totally shifts the color weighting, trading positions makes a mirror, and copying/smoothing small groups of pixels enlarges/reduces the image.

The hardest part of this program for me was remembering the row goes before the column in a two dimensional Java array. Making sure my effects worked on non-square images helped me see when I accidentally flipped those dimensions. I finished all of the required work for this lab. I did not complete any optional extra credit tasks.

## 4. Testing

I tested the effects with pixel 2-D matrices that I made and I made the expected results as well.
The JUnit tests created pixels for white, black, red, green, blue, and gray, applied one effect at a time, and compared every color value in the actual answer with the expected answer.

- I used simple colors for the NoColor and OnlyColor tests. So, removing red from a red pixel should produce black, while removing red from a green pixel should leave it green.
- The reflection tests use uneven patterns and non-square grids. This allowed me to see if the code flipped the correct direction and let me see if a row and a column got mixed up in the process.
- The Grow test checked to see that each starting pixel becomes the expected 2-by-2 block. The Shrink test confirms both the smaller size and the location of each averaged block.
- The Threshold test uses the normal cutoff of 127 because that was suggested in the Project Spec and I created a params arrayList in the test case, used it as an argument when I called the .apply method on pixels and params.

My cool images I produced:

(No Red) ft. The Weeknd.

(No Blue) ft. Niagara Falls